DAA: LAB 1

Analysis of Arrays and Linked Lists

Name: Sameer Gupta
Roll No: 252010036
Date: February 15, 2026

1 Array,Singly linked list and Doubly linked list :

```
#include <bits/stdc++.h>
using namespace std;
struct DNode
{
    DNode *next;
    DNode *prev;
    int val;
    DNode()
    {
        this->val = 0;
        this->next = NULL;
        this->prev = NULL;
    }
    DNode(int val)
    {
        this->val = val;
        this->next = NULL;
        this->prev = NULL;
    }
    DNode(int val, DNode *next, DNode *prev)
    {
        this->val = val;
        this->next = next;
        this->prev = prev;
    }
};
struct SNode
{
    SNode *next;
    int val;
    SNode()
    {
        this->val = 0;
        next = NULL;
    }
    SNode(int val)
    {
        this->val = val;
        this->next = NULL;
    }
    SNode(int val, SNode *next)
    {
        this->val = val;
        this->next = next;
```

```
    }
};
class Array
{
private:
    int *arr;
    int sizeArr;
    int capacity;

public:
    Array()
    {
        sizeArr = 0;
        capacity = 100;
        arr = new int[capacity];
    }
    ~Array()
    {
        delete[] arr;
    }
    void printArray()
    {
        if (sizeArr == 0)
        {
            cout << "Array is empty." << endl;
            return;
        }
        cout << "[ ";
        for (int i = 0; i < sizeArr; i++)
        {
            cout << arr[i] << " ";
        }
        cout << "]" << endl;
    }
    void push_back(int num)
    {
        if (sizeArr >= capacity)
        {
            cout << "Overflow: Array is full!" << endl;
            return;
        }
        arr[sizeArr] = num;
        sizeArr++;
    }
    void insertAtKthPosition(int num, int k)
    {
        if (k < 0 || k > sizeArr)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        if (sizeArr >= capacity)
        {
            cout << "Overflow: Array is full!" << endl;
```

```
            return;
        }
        for (int i = sizeArr - 1; i >= k; i--)
        {
            arr[i + 1] = arr[i];
        }

        arr[k] = num;
        sizeArr++;
    }
    void pop_back()
    {
        if (sizeArr <= 0)
        {
            cout << "Underflow: Array is empty!" << endl;
            return;
        }
        sizeArr--;
    }
    void deleteFromKthPosition(int k)
    {
        if (k < 0 || k >= sizeArr)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        for (int i = k; i < sizeArr - 1; i++)
        {
            arr[i] = arr[i + 1];
        }
        sizeArr--;
    }
    bool isPresent(int num)
    {
        for (int i = 0; i < sizeArr; i++)
        {
            if (arr[i] == num)
                return true;
        }
        return false;
    }
    void updateKthElement(int num, int k)
    {
        if (k < 0 || k >= sizeArr)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        arr[k] = num;
    }
    void reverseArray()
    {
        int i = 0, j = sizeArr - 1;
        while (i < j)
```

```
        {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }
    int getSize()
    {
        return sizeArr;
    }
};
class SinglyLinkedList
{
public:
    SinglyLinkedList()
    {
    }
    void traverseLL(SNode *head)
    {
        SNode *temp = head;
        while (temp != NULL)
        {
            cout << temp->val << " ";
            temp = temp->next;
        }
        cout << endl;
    }
    SNode *arrayToLL(vector<int> &arr)
    {
        if (arr.size() == 0)
            return NULL;
        SNode *head = new SNode(arr[0]);
        SNode *temp = head;
        for (int i = 1; i < arr.size(); i++)
        {
            SNode *newNode = new SNode(arr[i]);
            temp->next = newNode;
            temp = newNode;
        }
        return head;
    }
    SNode *insertAtKthPosition(SNode *head, int k, int num)
    {
        if (k < 1)
            return head;
        if (k == 1)
        {
            SNode *newHead = new SNode(num);
            newHead->next = head;
            return newHead;
        }
        int i = 0;
        SNode *temp = head;
        while (temp != NULL)
```

```
        {
            i++;
            if (i == k - 1)
            {
                SNode *newNode = new SNode(num);
                newNode->next = temp->next;
                temp->next = newNode;
                break;
            }
            temp = temp->next;
        }
        return head;
    }
    bool isPresentInLL(SNode *head, int num)
    {
        if (head == NULL)
            return false;
        SNode *temp = head;
        while (temp != NULL)
        {
            if (temp->val == num)
                return true;
            temp = temp->next;
        }
        return false;
    }
    SNode *deleteFromKthPosition(SNode *head, int k)
    {
        if (k < 1 || head == NULL)
            return head;
        if (k == 1)
        {
            SNode *toDelete = head;
            head = head->next;
            delete toDelete;
            return head;
        }
        SNode *temp = head;
        int i = 0;
        while (temp)
        {
            i++;
            if (i == k - 1)
            {
                if (temp->next != NULL)
                {
                    SNode *toDelete = temp->next;
                    temp->next = temp->next->next;
                    delete toDelete; // FIX: Free memory
                }
                break;
            }
            temp = temp->next;
        }
```

```
        return head;
    }
    void updateKthElement(SNode *head, int k, int num)
    {
        int i = 0;
        SNode *temp = head;
        while (temp)
        {
            i++;
            if (i == k)
            {
                temp->val = num;
                break;
            }
            temp = temp->next;
        }
    }
    SNode *reverseLL(SNode *head)
    {
        SNode *front = head, *curr = head, *prev = NULL;
        while (curr)
        {
            front = front->next;
            curr->next = prev;
            prev = curr;
            curr = front;
        }
        return prev;
    }
};
class DoublyLinkedList
{
public:
    DNode *arrayToDLL(vector<int> &arr)
    {
        if (arr.empty())
            return NULL;
        DNode *head = new DNode(arr[0]);
        DNode *temp = head;
        for (size_t i = 1; i < arr.size(); i++)
        {
            DNode *newNode = new DNode(arr[i]);
            newNode->prev = temp;
            temp->next = newNode;
            temp = newNode;
        }
        return head;
    }
    void traverseDLL(DNode *head)
    {
        cout << "List: ";
        DNode *temp = head;
        while (temp != NULL)
        {
```

```
            cout << temp->val << " ";
            temp = temp->next;
        }
        cout << endl;
    }
    DNode *insertAtKthPosition(DNode *head, int k, int num)
    {
        if (k < 1)
            return head;
        if (k == 1)
        {
            DNode *newHead = new DNode(num);
            newHead->next = head;
            if (head != NULL)
                head->prev = newHead;
            return newHead;
        }
        int i = 0;
        DNode *temp = head;
        while (temp != NULL)
        {
            i++;
            if (i == k - 1)
            {
                DNode *newNode = new DNode(num);
                newNode->next = temp->next;
                newNode->prev = temp;
                if (temp->next != NULL)
                {
                    temp->next->prev = newNode;
                }
                temp->next = newNode;
                break;
            }
            temp = temp->next;
        }
        return head;
    }
    bool isPresentInDLL(DNode *head, int num)
    {
        DNode *temp = head;
        while (temp != NULL)
        {
            if (temp->val == num)
                return true;
            temp = temp->next;
        }
        return false;
    }
    DNode *deleteFromKthPosition(DNode *head, int k)
    {
        if (head == NULL || k < 1)
            return head;
        // Case: Delete Head
```

```
        if (k == 1)
        {
            DNode *temp = head;
            head = head->next;
            if (head != NULL)
                head->prev = NULL;
            delete temp;
            return head;
        }
        DNode *temp = head;
        int i = 0;
        while (temp != NULL)
        {
            i++;
            if (i == k)
            {
                DNode *prevNode = temp->prev;
                DNode *nextNode = temp->next;
                if (prevNode != NULL)
                    prevNode->next = nextNode;
                if (nextNode != NULL)
                    nextNode->prev = prevNode;
                delete temp;
                break;
            }
            temp = temp->next;
        }
        return head;
    }
    void updateKthElement(DNode *head, int k, int num)
    {
        int i = 0;
        DNode *temp = head;
        while (temp)
        {
            i++;
            if (i == k)
            {
                temp->val = num;
                break;
            }
            temp = temp->next;
        }
    }
    DNode *reverseDLL(DNode *head)
    {
        if (head == NULL || head->next == NULL)
            return head;
        DNode *temp = head;
        DNode *last = NULL;
        while (temp != NULL)
        {
            last = temp->prev;
            temp->prev = temp->next;
```

```
            temp->next = last;
            temp = temp->prev;
        }
        if (last != NULL)
        {
            head = last->prev;
        }
        return head;
    }
};
int main()
{
    cout << endl;
    cout << "-----------------------------------------------------------------" << endl;
    cout << "------------------------------ARRAY------------------------------" << endl;
    cout << "-----------------------------------------------------------------" << endl;
    cout << endl;

    Array myArray;
    cout << "--- Initializing Array ---" << endl;
    myArray.push_back(8);
    myArray.push_back(9);
    myArray.push_back(2);
    myArray.push_back(1);
    cout << "Current Array: ";
    myArray.printArray();
    cout << "\n--- Insertion (Insert 10 at Index 1) ---" << endl;
    myArray.insertAtKthPosition(10, 1);
    cout << "After Insertion: ";
    myArray.printArray();
    cout << "\n--- Pop Back (Remove last element) ---" << endl;
    myArray.pop_back();
    cout << "After Pop Back: ";
    myArray.printArray();
    cout << "\n--- Delete (Delete element at Index 1) ---" << endl;
    myArray.deleteFromKthPosition(1);
    cout << "After Deletion: ";
    myArray.printArray();
    cout << "\n--- Update (Change element at Index 1 to 12) ---" << endl;
    myArray.updateKthElement(12, 1);
    cout << "After Update: ";
    myArray.printArray();
    cout << "\n--- Reverse Array ---" << endl;
    myArray.reverseArray();
    cout << "After Reverse: ";
    myArray.printArray();

    cout << endl;
```

```
    cout << "-----------------------------------------------------------------" << endl;
    cout << "------------------------Singly Linked List------------------------" << endl;
    cout << "-----------------------------------------------------------------" << endl;
    cout << endl;

    vector<int> arr1{1, 2, 3, 4, 5};
    SinglyLinkedList sll;
    cout << "--- 1. Creation ---" << endl;
    SNode *shead = sll.arrayToLL(arr1);
    sll.traverseLL(shead);
    cout << "\n--- 2. Insertion ---" << endl;
    cout << "Inserting 10 at position 2..." << endl;
    shead = sll.insertAtKthPosition(shead, 2, 10);
    sll.traverseLL(shead);
    cout << "\n--- 3. Search ---" << endl;
    int num1 = 10;
    if (sll.isPresentInLL(shead, num1))
        cout << num1 << " is present." << endl;
    else
        cout << num1 << " is not present." << endl;
    cout << "\n--- 4. Deletion ---" << endl;
    cout << "Deleting node at position 3..." << endl;
    shead = sll.deleteFromKthPosition(shead, 3);
    sll.traverseLL(shead);
    cout << "\n--- 5. Update ---" << endl;
    cout << "Updating node at position 2 to 67..." << endl;
    sll.updateKthElement(shead, 2, 67);
    sll.traverseLL(shead);
    cout << "\n--- 6. Reverse ---" << endl;
    cout << "Reversing the list..." << endl;
    shead = sll.reverseLL(shead);
    sll.traverseLL(shead);

    cout << endl;
    cout << "-----------------------------------------------------------------" << endl;
    cout << "------------------------Doubly Linked List------------------------" << endl;
    cout << "-----------------------------------------------------------------" << endl;
    cout << endl;

    DoublyLinkedList dll;
    vector<int> arr2{1, 2, 3, 4, 5};
    cout << "--- 1. Creation ---" << endl;
    DNode *dhead = dll.arrayToDLL(arr2);
    dll.traverseDLL(dhead);
```

```
    cout << "\n--- 2. Insertion ---" << endl;
    cout << "Inserting 10 at position 2..." << endl;
    dhead = dll.insertAtKthPosition(dhead, 2, 10);
    dll.traverseDLL(dhead);
    cout << "\n--- 3. Search ---" << endl;
    int num2 = 10;
    if (dll.isPresentInDLL(dhead, num2))
        cout << num2 << " is found." << endl;
    else
        cout << num2 << " is not found." << endl;
    cout << "\n--- 4. Deletion ---" << endl;
    cout << "Deleting node at position 3..." << endl;
    dhead = dll.deleteFromKthPosition(dhead, 3);
    dll.traverseDLL(dhead);
    cout << "\n--- 5. Update ---" << endl;
    cout << "Updating node at position 2 to 67..." << endl;
    dll.updateKthElement(dhead, 2, 67);
    dll.traverseDLL(dhead);
    cout << "\n--- 6. Reverse ---" << endl;
    cout << "Reversing the list..." << endl;
    dhead = dll.reverseDLL(dhead);
    dll.traverseDLL(dhead);
    return 0;
}
```

```
-------------------------------------------------------------
----------------------------ARRAY----------------------------
-------------------------------------------------------------

--- Initializing Array ---
Current Array: [ 8 9 2 1 ]

--- Insertion (Insert 10 at Index 1) ---
After Insertion: [ 8 10 9 2 1 ]

--- Pop Back (Remove last element) ---
After Pop Back: [ 8 10 9 2 ]

--- Delete (Delete element at Index 1) ---
After Deletion: [ 8 9 2 ]

--- Update (Change element at Index 1 to 12) ---
After Update: [ 8 12 2 ]

--- Reverse Array ---
After Reverse: [ 2 12 8 ]
```

Figure 1: Array output

```
-----------------------------------------------------------------
-----------------------Singly Linked List-----------------------
-----------------------------------------------------------------

--- 1. Creation ---
1 2 3 4 5

--- 2. Insertion ---
Inserting 10 at position 2...
1 10 2 3 4 5

--- 3. Search ---
10 is present.

--- 4. Deletion ---
Deleting node at position 3...
1 10 3 4 5

--- 5. Update ---
Updating node at position 2 to 67...
1 67 3 4 5

--- 6. Reverse ---
Reversing the list...
5 4 3 67 1
```

Figure 2: Singly Linked List output

```
-----------------------------------------------------------------
-----------------------Doubly Linked List-----------------------
-----------------------------------------------------------------

--- 1. Creation ---
List: 1 2 3 4 5

--- 2. Insertion ---
Inserting 10 at position 2...
List: 1 10 2 3 4 5

--- 3. Search ---
10 is found.

--- 4. Deletion ---
Deleting node at position 3...
List: 1 10 3 4 5

--- 5. Update ---
Updating node at position 2 to 67...
List: 1 67 3 4 5

--- 6. Reverse ---
Reversing the list...
List: 5 4 3 67 1
```

Figure 3: Doubly Linked List output

DAALab 1

Roll No: 252010036
Name: Sameer Gupta

Aim:
To study and implement operations on linear data structures - array, singly linked list & doubly linked list.

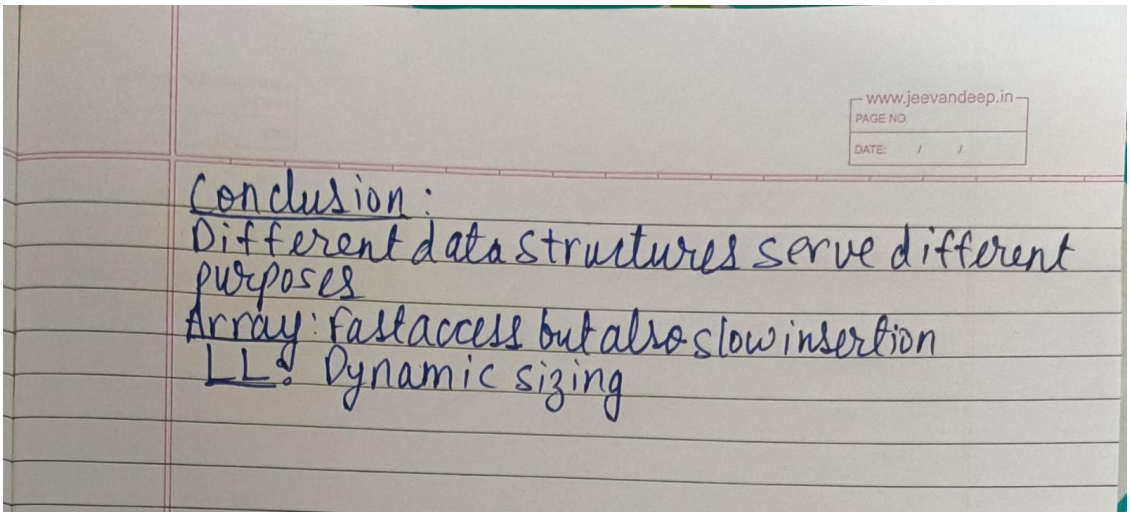
Arrays:
A linear data structure that stores elements in contiguous memory locations. It allows random access using an index.

Singly Linked List:
A collection of nodes where each node contains data and a pointer to the next node. It allows for dynamic memory allocation.

Doubly Linked List:
An extention to SLL, where each node contains two pointers, one to the next node & one to the previous node, allowing bidirectional traversal.

op	Array	SLL	DLL
insert	O(N)	O(N)	O(N)
Delete	O(N)	O(N)	O(N)
Search	O(N)	O(N)	O(N)
Update	O(1)	O(N)	O(N)

Figure 4: Theory 1.1



Conclusion:
Different data structures serve different purposes
Array: Fast access but also slow insertion
LL: Dynamic sizing

Figure 5: Theory 1.2